

Tarea 11 - Programacion SIG: Vectorizacion y DataFrames

Jesus Enrique Flores Riera

2026-05-22

Table of contents

1	Introduccion	1
2	Ejercicio 1: Vectorizacion y algebra espacial (Haversine)	2
2.1	Enunciado	2
2.2	Solucion en Python	2
2.3	Solucion en R	3
2.4	Solucion en Julia	3
3	Ejercicio 2: Limpieza, filtros y agregacion tabular	4
3.1	Enunciado	4
3.2	Solucion en Python	4
3.3	Solucion en R	5
3.4	Solucion en Julia	5
4	Reflexion argumentativa	6
4.1	Que aprendi sobre programacion SIG con vectorizacion	6
4.2	Diferencias entre Python, R y Julia	6
4.3	Manejo de valores faltantes en datos ambientales	6
4.4	Conclusion personal	7

1 Introduccion

Esta tarea aborda dos ejercicios fundamentales en la programacion aplicada a Sistemas de Informacion Geografica (SIG): la vectorizacion de operaciones matematicas sobre coordenadas geograficas y el procesamiento tabular de datos ambientales. Se implementan soluciones en tres lenguajes: **Python**, **R** y **Julia**.

2 Ejercicio 1: Vectorizacion y algebra espacial (Haversine)

2.1 Enunciado

La formula de Haversine permite calcular la gran distancia sobre la esfera terrestre dadas dos coordenadas en latitud y longitud. Tu mision es crear una funcion vectorizada llamada `calcular_distancias` que reciba como argumento el DataFrame `df` de censos y calcule la distancia en kilometros desde **Bogota** (Lat: 4.7110, Lon: -74.0721) a todas las demas ciudades de la tabla, agregando el resultado en una nueva columna llamada `Distancia_Bogota`. Aplica la matematica directamente sobre las columnas completas del DataFrame, evitando ciclos `for`.

2.2 Solucion en Python

```
import numpy as np
import pandas as pd

df = pd.DataFrame({
    'Ciudad': ['Bogota', 'Medellin', 'Cali', 'Barranquilla', 'Cartagena',
               'Cucuta', 'Bucaramanga', 'Pereira', 'Manizales', 'Ibague'],
    'Latitud': [4.7110, 6.2442, 3.4516, 10.9685, 10.3910,
                7.8939, 7.1254, 4.8133, 5.0703, 4.4389],
    'Longitud': [-74.0721, -75.5812, -76.5320, -74.7813, -75.4794,
                 -72.5078, -73.1198, -75.6961, -75.5138, -75.2322],
    'Poblacion': [7181469, 2533424, 2227642, 1274250, 1028736,
                  750015, 582984, 474335, 434403, 563822]
})

def calcular_distancias(df: pd.DataFrame) -> pd.DataFrame:
    R = 6371.0
    lat_bog = np.radians(4.7110)
    lon_bog = np.radians(-74.0721)
    lat2 = np.radians(df['Latitud'].values)
    lon2 = np.radians(df['Longitud'].values)
    dlat = lat2 - lat_bog
    dlon = lon2 - lon_bog
    a = np.sin(dlat/2)**2 + np.cos(lat_bog)*np.cos(lat2)*np.sin(dlon/2)**2
    c = 2 * np.arctan2(np.sqrt(a), np.sqrt(1 - a))
    df = df.copy()
    df['Distancia_Bogota'] = np.round(R * c, 2)
    return df

df = calcular_distancias(df)
print(df[['Ciudad', 'Distancia_Bogota']].to_string(index=False))
```

2.3 Solucion en R

```
library(dplyr)

df <- data.frame(
  Ciudad      = c('Bogota','Medellin','Cali','Barranquilla','Cartagena',
                  'Cucuta','Bucaramanga','Pereira','Manizales','Ibague'),
  Latitud     = c(4.7110, 6.2442, 3.4516, 10.9685, 10.3910,
                  7.8939, 7.1254, 4.8133, 5.0703, 4.4389),
  Longitud    = c(-74.0721,-75.5812,-76.5320,-74.7813,-75.4794,
                  -72.5078,-73.1198,-75.6961,-75.5138,-75.2322),
  Poblacion   = c(7181469,2533424,2227642,1274250,1028736,
                  750015,582984,474335,434403,563822)
)

calcular_distancias <- function(df) {
  R      <- 6371.0
  lat_bog <- 4.7110 * pi / 180
  lon_bog <- -74.0721 * pi / 180
  lat2 <- df$Latitud * pi / 180
  lon2 <- df$Longitud * pi / 180
  dlat <- lat2 - lat_bog
  dlon <- lon2 - lon_bog
  a <- sin(dlat/2)^2 + cos(lat_bog)*cos(lat2)*sin(dlon/2)^2
  c <- 2 * atan2(sqrt(a), sqrt(1-a))
  df$Distancia_Bogota <- round(R * c, 2)
  return(df)
}

df <- calcular_distancias(df)
print(df[, c('Ciudad','Distancia_Bogota')])
```

2.4 Solucion en Julia

```
using DataFrames, Statistics

df = DataFrame(
  Ciudad      = ["Bogota","Medellin","Cali","Barranquilla","Cartagena",
                  "Cucuta","Bucaramanga","Pereira","Manizales","Ibague"],
  Latitud     = [4.7110, 6.2442, 3.4516, 10.9685, 10.3910,
                  7.8939, 7.1254, 4.8133, 5.0703, 4.4389],
  Longitud    = [-74.0721,-75.5812,-76.5320,-74.7813,-75.4794,
                  -72.5078,-73.1198,-75.6961,-75.5138,-75.2322],
  Poblacion   = [7181469,2533424,2227642,1274250,1028736,
                  750015,582984,474335,434403,563822]
```

```

)

function calcular_distancias(df::DataFrame)::DataFrame
    R      = 6371.0
    lat_bog = deg2rad(4.7110)
    lon_bog = deg2rad(-74.0721)
    lat2 = deg2rad.(df.Latitud)
    lon2 = deg2rad.(df.Longitud)
    dlat = lat2 .- lat_bog
    dlon = lon2 .- lon_bog
    a = sin.(dlat./2).^2 .+ cos(lat_bog).*cos.(lat2).*sin.(dlon./2).^2
    c = 2 .* atan.(sqrt.(a), sqrt.(1 .- a))
    result = copy(df)
    result.Distancia_Bogota = round.(R .* c, digits=2)
    return result
end

df = calcular_distancias(df)
println(select(df, :Ciudad, :Distancia_Bogota))

```

3 Ejercicio 2: Limpieza, filtros y agregacion tabular

3.1 Enunciado

Genera un DataFrame con datos de estaciones meteorologicas (Paramo, Valle, Costa, Selva, Sabana). Los sensores de la Costa y la Sabana fallaron: rellena sus valores nulos de precipitacion con la media de las estaciones validas. Filtra las estaciones con elevacion estrictamente mayor a 1000 m en un nuevo DataFrame `estaciones_altas` y muestra la precipitacion promedio anual de ese subconjunto.

3.2 Solucion en Python

```

import pandas as pd

estaciones = pd.DataFrame({
    'Estacion':      ['Paramo', 'Valle', 'Costa', 'Selva', 'Sabana'],
    'Elevacion':     [3200, 1500, 15, 200, 2600],
    'Precipitacion': [850.5, 1200.0, None, 3000.5, None]
})

media_valida = estaciones['Precipitacion'].mean()
estaciones['Precipitacion'] = estaciones['Precipitacion'].fillna(media_valida)

```

```
estaciones_altas = estaciones[estaciones['Elevacion'] > 1000].copy()
prom_altas = estaciones_altas['Precipitacion'].mean()
print(f'Precipitacion promedio en estaciones altas: {prom_altas:.2f} mm')
```

3.3 Solucion en R

```
library(dplyr)

estaciones <- data.frame(
  Estacion      = c('Paramo','Valle','Costa','Selva','Sabana'),
  Elevacion     = c(3200, 1500, 15, 200, 2600),
  Precipitacion = c(850.5, 1200.0, NA, 3000.5, NA)
)

media_valida <- mean(estaciones$Precipitacion, na.rm = TRUE)
estaciones$Precipitacion[is.na(estaciones$Precipitacion)] <- media_valida

estaciones_altas <- estaciones %>% filter(Elevacion > 1000)
prom_altas <- mean(estaciones_altas$Precipitacion)
cat(sprintf('Precipitacion promedio en estaciones altas: %.2f mm\n', prom_altas))
```

3.4 Solucion en Julia

```
using DataFrames, Statistics

estaciones = DataFrame(
  Estacion      = ["Paramo","Valle","Costa","Selva","Sabana"],
  Elevacion     = [3200, 1500, 15, 200, 2600],
  Precipitacion = Union{Float64,Missing}[850.5, 1200.0, missing, 3000.5, missing]
)

media_valida = mean(skipmissing(estaciones.Precipitacion))
estaciones.Precipitacion = coalesce.(estaciones.Precipitacion, media_valida)

estaciones_altas = filter(row -> row.Elevacion > 1000, estaciones)
prom_altas = mean(estaciones_altas.Precipitacion)
println("Precipitacion promedio en estaciones altas: ", round(prom_altas, digits=2), " mm")
```

4 Reflexion argumentativa

4.1 Que aprendi sobre programacion SIG con vectorizacion

La realizacion de esta tarea consolido varios principios esenciales en la programacion orientada a datos espaciales. El primero y mas relevante es la **vectorizacion**: en lugar de recorrer fila por fila un DataFrame con un ciclo `for`, se aplica la operacion matematica directamente sobre arreglos o columnas completas. Esto no es solo un estilo de codigo mas elegante, sino una diferencia de rendimiento real, ya que las bibliotecas subyacentes (NumPy, R base, Julia) delegan esas operaciones a rutinas compiladas en C/Fortran que aprovechan instrucciones SIMD del procesador.

La formula de Haversine es un ejemplo clasico de algebra geodesica vectorizable: convertir grados a radianes, calcular diferencias angulares y aplicar trigonometria son operaciones identicas para cada par de coordenadas, lo que las hace perfectamente paralelizables en arreglos. Trabajar con coordenadas geograficas en DataFrames es el primer paso hacia analisis SIG completos en Python (GeoPandas), R (`sf`) o Julia (GeoDataFrames.jl).

4.2 Diferencias entre Python, R y Julia

Aspecto	Python (NumPy/Pandas)	R (dplyr/base)	Julia (DataFrames.jl)
Sintaxis vectorial	<code>np.sin(arr)</code>	<code>sin(vec)</code> nativo	<code>sin.(vec)</code> broadcast explicito
Nulos	<code>None</code> / <code>np.nan</code>	<code>NA</code>	<code>missing</code>
Imputacion	<code>fillna(media)</code>	<code>is.na()</code> + asignacion	<code>coalesce.()</code>
Filtro tabular	<code>df[condicion]</code>	<code>filter(df, cond)</code>	<code>filter(fn, df)</code>
Rendimiento	Bueno (C bajo NumPy)	Bueno (BLAS/Fortran)	Excelente (JIT nativo)
Curva de aprendizaje	Moderada	Baja para estadistica	Alta, pero muy expresiva
Ecosistema SIG	GeoPandas, Shapely	<code>sf</code> , <code>terra</code> , <code>stars</code>	GeoDataFrames.jl

Python sobresale por su ecosistema maduro y la integracion con herramientas de machine learning (scikit-learn, PyTorch). R es el lenguaje natural del estadistico y ofrece el paquete `sf` que maneja datos espaciales de forma idiomática con proyecciones CRS integradas. Julia destaca por su filosofia de ‘velocidad de C con sintaxis de alto nivel’: el broadcast explicito con el punto `(.)` hace transparente cuando una operacion es escalar y cuando es vectorial, lo que reduce errores conceptuales al trabajar con arreglos geograficos.

4.3 Manejo de valores faltantes en datos ambientales

El Ejercicio 2 replicó un escenario muy comun en SIG y teledeteccion: sensores que fallan y generan huecos en series temporales o tablas de atributos. La estrategia de imputacion por media es simple pero efectiva para datos sin tendencia marcada. En contextos mas avanzados (series temporales de precipitacion) se emplearian interpolacion espacial (kriging), metodos de interpolacion temporal o

modelos predictivos. El ejercicio demostro ademas la importancia de aplicar el filtro **despues** de la imputacion: si se filtrara primero, la media se calcularia solo con las estaciones altas, sesgando la estimacion.

4.4 Conclusion personal

Implementar la misma solucion en tres lenguajes distintos fue una experiencia enriquecedora: revela que los conceptos (vectorizacion, imputacion, filtrado) son universales, pero cada lenguaje tiene su idioma propio. Dominar estos patrones en Python, R y Julia permite elegir la herramienta mas adecuada segun el contexto del proyecto SIG, sea un script de procesamiento masivo de imagenes satelitales en Python, un analisis estadistico en R o un modelo de simulacion de alto rendimiento en Julia.